

```

"""
bot_engine.py
Core strategy logic + paper trading simulation,
with state saved to a
local SQLite database so it survives restarts on
the cloud server.

Strategy: EMA(9) x EMA(21) crossover confirmed by
RSI(14), intraday,
fixed stop-loss / target, auto square-off before
market close.
"""

import sqlite3
import numpy as np
import pandas as pd
from datetime import datetime, time
import pytz

IST = pytz.timezone("Asia/Kolkata")
DB_PATH = "trading_bot.db"

# ----- STRATEGY SETTINGS -----
--
EMA_FAST = 9
EMA_SLOW = 21
RSI_PERIOD = 14
RSI_BUY_THRESHOLD = 50
RSI_SELL_THRESHOLD = 50
STOP_LOSS_PCT = 0.30
TARGET_PCT = 0.60
SQUARE_OFF_TIME = time(15, 15)
MARKET_OPEN = time(9, 15)
MARKET_CLOSE = time(15, 30)
CAPITAL_PER_TRADE = 100000

```

```

TICKERS = {
    "NIFTY": "^NSEI",
    "BANKNIFTY": "^NSEBANK",
}

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS trades (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            symbol TEXT,
            side TEXT,
            entry_time TEXT,
            entry_price REAL,
            exit_time TEXT,
            exit_price REAL,
            pnl_pct REAL,
            pnl_rupees REAL,
            exit_reason TEXT,
            status TEXT
        )
    """)
    c.execute("""
        CREATE TABLE IF NOT EXISTS status_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            symbol TEXT,
            message TEXT
        )
    """)
    conn.commit()
    conn.close()

def log_status(symbol, message):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(

```

```

        "INSERT INTO status_log (timestamp, symbol,
message) VALUES (?, ?, ?)",
        (datetime.now(IST).isoformat(), symbol,
message),
    )
    conn.commit()
    conn.close()

def calculate_rsi(series, period=14):
    delta = series.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()
    rs = avg_gain / avg_loss.replace(0, np.nan)
    return 100 - (100 / (1 + rs))

def fetch_latest_data(ticker):
    import yfinance as yf
    df = yf.download(ticker, period="5d",
interval="5m", progress=False)
    if df.empty:
        return None
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = [c[0] for c in df.columns]
    if df.index.tz is None:
        df = df.tz_localize("UTC")
    df = df.tz_convert(IST)
    df["ema_fast"] = df["Close"].ewm(span=EMA_FAST,
adjust=False).mean()
    df["ema_slow"] = df["Close"].ewm(span=EMA_SLOW,
adjust=False).mean()
    df["rsi"] = calculate_rsi(df["Close"],
RSI_PERIOD)
    return df

def get_open_position(symbol):

```

```

conn = sqlite3.connect(DB_PATH)
c = conn.cursor()
c.execute(
    "SELECT id, side, entry_time, entry_price
FROM trades WHERE symbol=? AND status='OPEN'",
    (symbol,),
)
row = c.fetchone()
conn.close()
if row:
    return {"id": row[0], "side": row[1],
"entry_time": row[2], "entry_price": row[3]}
return None

def open_position(symbol, side, price, ts):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(
        "INSERT INTO trades (symbol, side,
entry_time, entry_price, status) VALUES (?, ?, ?,
?, 'OPEN')",
        (symbol, side, ts.isoformat(), price),
    )
    conn.commit()
    conn.close()
    log_status(symbol, f"Opened {side} at
{price:.2f}")

def close_position(trade_id, symbol, exit_price,
ts, reason, pnl_pct, pnl_rupees):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""
        UPDATE trades SET exit_time=?,
exit_price=?, exit_reason=?, pnl_pct=?,
pnl_rupees=?, status='CLOSED'
        WHERE id=?
        """, (ts.isoformat(), exit_price, reason,

```

```

pnl_pct, pnl_rupees, trade_id))
    conn.commit()
    conn.close()
    log_status(symbol, f"Closed ({reason}) at
{exit_price:.2f}, P&L Rs {pnl_rupees:.2f}")

def within_market_hours(now_ist):
    t = now_ist.time()
    return MARKET_OPEN <= t <= MARKET_CLOSE

def check_and_trade(symbol_key):
    """Called every ~5 minutes by the scheduler
    during market hours."""
    ticker = TICKERS[symbol_key]
    now = datetime.now(IST)

    if now.weekday() >= 5: # Saturday/Sunday
        return
    if not within_market_hours(now):
        return

    df = fetch_latest_data(ticker)
    if df is None or len(df) < EMA_SLOW + 2:
        log_status(symbol_key, "No data / not
        enough candles yet")
        return

    latest = df.iloc[-1]
    prev = df.iloc[-2]
    price = float(latest["Close"])

    position = get_open_position(symbol_key)

    # --- Manage existing position ---
    if position:
        if position["side"] == "BUY":
            pnl_pct = (price -
            position["entry_price"]) / position["entry_price"]

```

```

* 100
    else:
        pnl_pct = (position["entry_price"] -
price) / position["entry_price"] * 100

        exit_reason = None
        if now.time() >= SQUARE_OFF_TIME:
            exit_reason = "Intraday square-off"
        elif pnl_pct <= -STOP_LOSS_PCT:
            exit_reason = "Stop-loss hit"
        elif pnl_pct >= TARGET_PCT:
            exit_reason = "Target hit"

        if exit_reason:
            pnl_rupees = CAPITAL_PER_TRADE *
(pnl_pct / 100)
            close_position(position["id"],
symbol_key, price, now, exit_reason, pnl_pct,
pnl_rupees)
            return # one trade at a time per symbol

        # --- Look for new entry ---
        crossed_up = prev["ema_fast"] <=
prev["ema_slow"] and latest["ema_fast"] >
latest["ema_slow"]
        crossed_down = prev["ema_fast"] >=
prev["ema_slow"] and latest["ema_fast"] <
latest["ema_slow"]

        if crossed_up and latest["rsi"] >
RSI_BUY_THRESHOLD:
            open_position(symbol_key, "BUY", price,
now)
        elif crossed_down and latest["rsi"] <
RSI_SELL_THRESHOLD:
            open_position(symbol_key, "SELL", price,
now)
        else:
            log_status(symbol_key, f"No signal. Price
{price:.2f}, RSI {latest['rsi']:.1f}")

```

```

def run_all_symbols():
    for symbol_key in TICKERS:
        try:
            check_and_trade(symbol_key)
        except Exception as e:
            log_status(symbol_key, f"ERROR: {e}")

def get_dashboard_data():
    conn = sqlite3.connect(DB_PATH)
    trades_df = pd.read_sql_query("SELECT * FROM
trades ORDER BY id DESC", conn)
    status_df = pd.read_sql_query("SELECT * FROM
status_log ORDER BY id DESC LIMIT 30", conn)
    conn.close()

    closed = trades_df[trades_df["status"] ==
"CLOSED"] if not trades_df.empty else trades_df
    open_trades = trades_df[trades_df["status"] ==
"OPEN"] if not trades_df.empty else trades_df

    summary = {}
    for symbol_key in TICKERS:
        sym_closed = closed[closed["symbol"] ==
symbol_key] if not closed.empty else closed
        total = len(sym_closed)
        wins =
len(sym_closed[sym_closed["pnl_rupees"] > 0]) if
total else 0
        total_pnl = sym_closed["pnl_rupees"].sum()
if total else 0
        summary[symbol_key] = {
            "total_trades": total,
            "win_rate": round(wins / total * 100,
1) if total else 0,
            "total_pnl": round(total_pnl, 2),
        }

```

```
    return {
        "summary": summary,
        "open_trades":
open_trades.to_dict("records"),
        "closed_trades": closed.to_dict("records"),
        "status_log": status_df.to_dict("records"),
        "last_updated":
datetime.now(IST).strftime("%d-%b-%Y %H:%M:%S
IST"),
    }
```